

CSCE 671 HUMAN-COMPUTER INTERACTION · MODULE 10 · LECTURE NOTES

Interfaces for Programmers

APIs, errors, and configuration are user interfaces.

MODULE OUTCOMES

- Apply the course's principles to an API.
- Write an error message that helps.
- Explain what makes documentation usable.
- Design configuration so its state is visible.
- Evaluate a developer tool with the same methods.

1 An API is an interface

Principle	What it means in an API	The characteristic failure
Visibility	The available operations are discoverable from the type signature and the autocompletion.	A string parameter with undocumented permitted values, which is invisible by construction.
Feedback	Errors that say what went wrong and what to do (§2).	A null return, or a generic exception with no context.
Consistency	Argument order, naming, and error behaviour the same throughout.	Two related functions with reversed argument orders — which produces silent bugs.
Constraints	Types that make the misuse fail to compile.	Everything is a string or an int, so any value can be passed anywhere. See the note.
Good defaults	The safe and common thing happens with no arguments at all.	Insecure or slow unless explicitly configured (CSCE 701 Module 11 §3).
Reversibility	Operations that can be undone, or safely retried.	A partial write with no rollback and no way to tell what succeeded.

The constraints row is where types earn their keep — which is CSCE 713 Module 04 §4's parse-don't-validate argument arriving as a usability principle rather than a security one, and the convergence is worth noticing: a type that makes the invalid state unrepresentable is simultaneously a security control and a usability feature, for the same reason. The direct mapping is this module's whole argument: nothing new is required, only the recognition that programmers are users.

THE SIGNATURE IS THE DOCUMENTATION THAT IS ACTUALLY READ

A programmer reads the signature, the parameter names, and the autocompletion list — and reaches the prose documentation only when those have failed, which is Module 01 §2's non-reading finding appearing in a new population that likes to think it is exempt.

So a well-named function with a precise type is better documentation than a paragraph, because it is present at the point of use and the paragraph is in a browser tab that was never opened.

And a boolean parameter is a documentation failure: `process(data, true)` is unreadable at the call site, where an enumerated value — `process(data, Mode.Strict)` — would have been self-describing at no cost.

Which gives a concrete and mechanical rule: if the call site is not readable without consulting the definition, the signature is wrong — and that is a check you can apply in code review without any argument about taste, which makes it unusually actionable for this subject.

2 Error messages

```
WHAT HAPPENED
    specifically, in terms the caller recognises

WHAT THE PROGRAM WAS DOING
    the operation and the input, not the internal
    function name

WHY IT FAILED
    the actual condition, with the actual values

AND WHAT TO DO ABOUT IT
    the next step, or where to look

S0: "Configuration key 'timeout' must be a positive
integer; got 'none' in config.yaml line 14. Use 0 to
disable."

NOT: "Invalid configuration."
NOT: "Error: NullPointerException"
NOT: "Something went wrong. Try again."

AND INCLUDE THE VALUE. An error that does not say
what the offending input was forces the reader to
reproduce it, which is the most expensive thing an
error message can do.
```

Include the offending value — an error that omits it forces the reader to reproduce the failure in order to find out what it was objecting to, which is the most expensive failure an error message can have and is extremely common. The include-the-value rule is the single highest-return change available here, and it is a one-line edit per message. (With one exception worth stating: do not include a

secret — CSCE 711 Module 09 §4's logging rule applies, and the fix is to include the length or the shape rather than the value.)

AND AN ERROR MESSAGE IS READ AT THE WORST POSSIBLE MOMENT

The reader is interrupted, frustrated, and under time pressure — which is exactly the state in which attention is least available (Module 03 §1) and in which a long message will not be read at all.

So it has to be scannable: the specific condition first, the supporting detail after, and no apology — 'oops, something went wrong' spends the reader's attention on nothing and delays the useful part.

And it must not blame the user, which is both **Module 01 §4's framing** and a practical matter: **'invalid input' when the input was entirely reasonable teaches nothing** and antagonises somebody who is already having a bad time.

Which makes error messages worth reviewing as deliberately as any other interface text — and they are typically written last, in a hurry, by whoever happened to hit the condition, which is exactly why they are poor and why reviewing them is high-return.

3 Documentation

- **A tutorial** gets a newcomer to a first working result. **Its job is a working result, not coverage** — and **completeness actively harms it**, because every additional option is a decision the newcomer cannot make yet.
- **A how-to guide** solves one specific stated problem for somebody who already understands the system — 'how do I do X' — and should assume the tutorial was read.
- **Reference** describes the surface exhaustively and **makes no attempt to teach** — **which is correct, and is what generated documentation is genuinely good at**, provided nobody mistakes it for the other three.
- **And explanation** conveys the conceptual model (Module 04) — why the system is organised as it is, what the central abstractions mean, and what it is not for. **Which is the kind most often missing and the most valuable**, because without it the reference is a list of names.
- **Conflating them produces documentation that teaches nobody and references nothing** — a tutorial interrupted by exhaustive option lists, or a reference that keeps trying to explain — **which describes most of the documentation that exists. Explanation is the kind most often missing**, and writing it is the thing a maintainer is uniquely positioned to do.

4 Configuration, and evaluation

A CONFIGURATION SYSTEM WHOSE EFFECTIVE STATE IS INVISIBLE CANNOT BE REASONED ABOUT

Values arrive from compiled-in defaults, a configuration file, an environment variable, a command-line flag, and sometimes a remote service — and **no interface shows which one won for any given setting**.

So the user cannot tell **why the system is behaving as it is, which is Module 01 §1's gulf of evaluation in its most consequential form**: not 'what did my action do' but 'what is this system's actual state', which is unanswerable.

And the fix is a single command that prints the effective configuration with each value's source — **which is an afternoon's work and transforms the debuggability of the whole system**, and is still absent from a great deal of widely used software.

Plus: validate at start-up rather than at point of use — **a typo in a setting should fail immediately and loudly, not three hours later when the relevant code path is first reached** — which is **Module 01 §4's make-the-error-visible applied to configuration** and is CSCE 713 Module 04 §4's parse-at-the-boundary in another guise.

- **Use the same methods. Watch five programmers try to accomplish a task with your library, without helping** (Module 07) — **which almost nobody does**, despite it being exactly as cheap and exactly as informative as for any other interface.

- **The findings are the same kinds: missing signifiers (an operation they could not find), absent feedback (an error that told them nothing), and model mismatches (they expected it to work differently)** — all diagnosed with Module 01 §1's vocabulary.
- **Watch what they search for**, because **the search query is the vocabulary they expected** and **your naming did not match it** — which is Module 04 §2's vocabulary point and is immediately actionable.
- **And read your own issue tracker as usability data** — **a recurring question is a design finding rather than a documentation gap**, and the fifth time it is asked is evidence rather than coincidence.
- **Which makes this the most actionable module in the course for most of this program's graduates**, since the interfaces they will design are APIs, error messages, configuration files, and tools. **A recurring question in your issue tracker is a design finding: answering it again is treating the symptom, and it is free data you already have.**

Where to see this taught differently

Original material only goes so far — hearing the same idea in another voice is how it sticks. These are free, and they cover this module specifically.

Resource	What it covers for this module
Myers & Stylos — Improving API usability (free)	§1 with the empirical studies — programmers watched using APIs, and what the failures were.
Procházka — Diátaxis documentation framework (free)	§3's four kinds — the clearest statement of the distinction and why conflating them fails.
Rust's error message guidelines, and the Elm compiler's approach (free)	§2 done well, deliberately — two projects that treated error messages as an interface and documented how.
Bloch — How to Design a Good API and Why it Matters (free talk)	§1 from a practitioner — and much of it maps onto §1's table without being framed that way.

Exercises

Do these in code, not on paper, unless the exercise says otherwise. The module is not finished until they run.

- 1 **Map all six principles** onto an API you maintain, and find a failure for each.
- 2 **Find a boolean parameter** in your own code and replace it with an enumerated type.
- 3 **Apply the call-site readability test** to twenty call sites.
- 4 **Rewrite five error messages** to include all four components.
- 5 **Check that each includes the offending value** and no secrets.
- 6 **Classify your own documentation** into the four kinds, and find which is missing.

- 7 **Write the explanation document** for something you maintain.
- 8 **Add a command that prints the effective configuration** with sources.
- 9 **Move your configuration validation to start-up** and see what it catches.
- 10 **Watch five programmers use your library** and record what they searched for.

Self-check

Answer these in writing, from memory, without looking back at the notes. If you cannot, you have read this module but not yet learned it — that is useful information, not a failure.

- 1 Map the six principles onto an API, with a failure for each.
- 2 Why does the constraints row connect to CSCE 713?
- 3 Why is the signature the documentation that is read?
- 4 Give the call-site readability rule.
- 5 Name the four components of an error message.
- 6 Why is including the value the highest-return change, and what is the exception?
- 7 Why is an error message read at the worst moment, and what follows?
- 8 Name the four kinds of documentation and which is most often missing.
- 9 Why can an invisible configuration state not be reasoned about?
- 10 How do you evaluate a developer tool, and what free data do you have?